Condicionales y Bucles

Condicionales (`th:if`, `th:unless`, `th:switch`, `th:case`)

En Thymeleaf, las estructuras condicionales permiten **mostrar u ocultar** partes del HTML según el valor de una variable del modelo..

`th:if`

Permite **mostrar un elemento solo si se cumple una condición**.

```
<p th:if="${usuario.edad >= 18}">Eres mayor de edad</p>
```

El párrafo se mostrará **solo si la condición es verdadera**.

`th:unless`

Hace lo contrario de `th:if`: se muestra si la condición **no** se cumple.

```
<p th:unless="${usuario.edad >= 18}">Eres menor de edad</p>
```

Usar `th:unless` puede hacer el código más legible cuando queremos mostrar el caso “contrario” sin necesidad de un `th:if` con una negación. Se puede ver como el else de un if.

`th:switch` y `th:case`

Sirven para evaluar una variable con **múltiples opciones** (similar a `switch` en Java).

```
<div th:switch="${usuario.rol}">
  <p th:case="'ADMIN'">Acceso de administrador</p>
  <p th:case="'USER'">Acceso de usuario</p>
  <p th:case="*">Rol no reconocido</p> <!-- * = caso por defecto -->
</div>
```

Condicionales rápidos y Operador Elvis

Condicionales contraídos

- **`cond ? a : b`** → ternario clásico, elige entre dos valores.
- **`expr ?: valor`** → Elvis, devuelve expr si no es nulo, si lo es usa valor alternativo.
- **`obj?.prop`** → null-safe, evita errores al acceder a propiedades de objetos nulos.

Operador ternario (if-else)

- Sintaxis:

  ```
  condición ? valor_si_verdadero : valor_si_falso
  ```

- Permite elegir entre dos valores según una condición.

Ejemplo:

```
<p th:text="${age >= 18} ? 'Mayor de edad' : 'Menor de
edad'">Texto</p>
```

Si **`age = 20`**, muestra: **Mayor de edad**.

Operador Elvis (`?:`)

- Es una forma abreviada del ternario.
- Devuelve el valor de la izquierda si **no es nulo**, en caso contrario devuelve el de la derecha.

Ejemplo:

```
<p th:text="${user.name} ?: 'Invitado'">Nombre</p>
```

Si **`user.name = null`**, muestra: **Invitado**.

Operador “null-safe” (`?.`)

- Evita errores cuando un objeto puede ser nulo.
- Si el objeto es nulo, la expresión devuelve `null` en lugar de un error.

Ejemplo:

```
<p th:text="${user?.address?.city}">Ciudad</p>
```

Si `user` es `null`, no rompe la plantilla (solo devuelve vacío).

Bucles (`th:each`)

La instrucción `th:each` permite **iterar sobre listas, arrays o colecciones** de objetos.

Bucle básico

Permite **iterar sobre colecciones o arrays.**

```
<ul>
  <li th:each="producto : ${productos}" th:text="${producto.nombre}"></li>
</ul>
```

Repite la etiqueta `<li>` por cada elemento en `productos`.

Acceder al índice

Se puede declarar una **variable de estado** opcional para acceder a información del bucle (índice, paridad, etc.):

```
<tr th:each="p, estado : ${productos}">
  <td th:text="${estado.index}">0</td>
  <td th:text="${p.nombre}">Nombre</td>
</tr>
```

Propiedades de la variable de estado:

- `index` → posición (empieza en 0)
- `count` → contador (empieza en 1)
- `size` → tamaño total de la colección
- `even` / `odd` → par o impar
- `first` / `last` → primer o último elemento

Bucle con condición interna

Puedes combinar `th:each` con `th:if` dentro del mismo bloque:

```
<li th:each="p : ${productos}" th:if="${p.stock > 0}">
  <span th:text="${p.nombre}"></span>
</li>
```

Iterar sobre mapas

Si el modelo contiene un `Map`, se pueden recorrer las **claves y valores**:

```
<ul>
  <li th:each="entrada : ${mapa}"
      th:text="${entrada.key + ' → ' + entrada.value}"></li>
</ul>
```

Notas importantes

- `th:each` **reemplaza el contenido** del elemento en cada iteración.
- Las variables del bucle **solo existen dentro del elemento** donde se declaran.
- Si la lista está vacía, el bloque simplemente **no se renderiza**.

Buenas prácticas

Usa `th:if` y `th:unless` para controlar la visibilidad, no para ejecutar lógica compleja.

Evita anidar demasiadas condiciones dentro de bucles: divide la lógica en el controlador cuando sea posible.

Aprovecha `th:switch` para situaciones con múltiples casos claros.

Usa la variable de estado en `th:each` para resaltar filas pares o numerar listas.

Recuerda que si la colección está vacía, el bloque simplemente **no se renderiza**

Ejercicios Básicos: Condicionales y Bucles en Thymeleaf

Ejercicio 1 – Mostrar un mensaje condicional

Objetivo: Practicar `th:if` y `th:unless`.

Crea una plantilla que reciba una variable `usuario` con una propiedad `edad`.
Muestra los siguientes mensajes:

- Si tiene 18 años o más: “Eres mayor de edad.”
- Si tiene menos de 18 años: “Eres menor de edad.”

Pistas:

- Usa `th:if` y `th:unless`.
- No uses JavaScript ni CSS.

Ejercicio 2 – Mostrar el rol de usuario

Objetivo: Usar `th:switch` y `th:case`.

Crea un fragmento de HTML que muestre un mensaje distinto según el rol del usuario (`ADMIN`, `USER` o cualquier otro valor).

Ejemplo esperado:

Rol: ADMIN → “Tienes acceso total al sistema.”

Rol: USER → “Tienes acceso limitado.”

Otro → “Rol no reconocido.”

Pistas:

- Usa `th:switch="${usuario.rol}"`.
- Incluye un caso por defecto con `th:case="*"`.

Ejercicio 3 – Listar productos con bucle

Objetivo: Practicar `th:each`.

Crea una lista de productos con nombre y precio.
Muestra todos los productos en una lista HTML `<ul>`, con el nombre y precio en cada elemento.

Ejemplo:

```
Manzana – 2.5 €

Portátil – 800 €

Camiseta – 9.99 €
```

Pistas:

- Usa `th:each="p : ${productos}"`.
- Muestra los valores con `th:text`.

Ejercicio 4 – Mostrar sólo productos en oferta

Objetivo: Combinar `th:each` con `th:if`.

Partiendo de la lista del ejercicio anterior, muestra **solo** los productos cuyo precio sea **menor que 10**.
Junto al nombre, añade la palabra “(Oferta)”.

Pistas:

- Usa `th:if="${p.precio < 10}"` dentro del bucle.

Ejercicio 5 – Numerar los productos

Objetivo: Usar la variable de estado del bucle.

Muestra todos los productos junto con su número de posición en la lista.
Ejemplo:

```
1. Manzana

2. Portátil

3. Camiseta
```

Pistas:

- Declara la variable de estado: `th:each="p, estado : ${productos}"`.
- Usa `estado.count` para mostrar el número.

Ejercicio 6 – Resaltar filas pares

Objetivo: Usar propiedades `even` y `odd`.

Muestra una tabla con los productos.
Las filas pares deben tener la clase CSS `"fila-par"`.

Pistas:

- Usa `th:classappend="${estado.even} ? 'fila-par' : ''"`.
- Crea un estilo simple:

```
.fila-par { background-color: #eee; }
```

Ejercicio 7 – Mostrar un mapa de países

Objetivo: Practicar bucles con mapas.

Recibe en el modelo un mapa `paises` donde la clave es el nombre del país y el valor es su capital.
Muestra la siguiente lista:

```
España → Madrid

Italia → Roma

Francia → París
```

Pistas:

- Usa `th:each="entrada : ${paises}"`.
- Accede con `${entrada.key}` y `${entrada.value}`.

Ejercicio 8 – Mostrar mensajes combinados

Objetivo: Combinar condiciones y bucles.

Recorre la lista de productos y muestra:

- "Disponible" si `p.stock > 0`.
- "Sin stock" si `p.stock == 0`.

Ejemplo:

```
Manzana – Disponible

Portátil – Sin stock
```

Pistas:

- Usa `th:each` con un `<span>` o `<p>` que tenga un `th:if` o `th:unless` dentro del bucle.